

# 【Python基础语法】参考

## 第0关

### 练习-打印皮卡丘-参考

请你使用\*\*print()函数\*\*和\*\*""\*\*将下面的皮卡丘打印出来。皮卡丘字符画可以直接复制步骤1里的。

```
...
    ^          /|
    /\7      <_/
    / |      / /
    | Z __,< /   /\
    |      \   /   }
    Y      ` /   /
    |● 、 ●   <= < /
    () ^      | \ <
    >- _-   | / /
    / ^     / /<| \ \
    \_/    (_/   | / /
    7              | /
    >-r-----_ _
...

```

### 题目讲解

- 1.复制题目要求打印的“皮卡丘”（直接复制下一步中的字符画即可）
  2. 用print('""')打印
- \*\*注意：标点符号都要用英文\*\***

### 【参考】

```
1 print(''
2     ^          /|
3     /\7      <_/
4     / |      / /
5     | Z __,< /   /\
6     |      \   /   }
7     Y      ` /   /
8     |● 、 ●   <= < /
9     () ^      | \ <
10    >- _-   | / /
11    / ^     / /<| \ \
12    \_/    (_/   | / /
13    7              | /

```

```
14     >-r-----\--_
15     ''')
```

## 练习-定个小目标-参考

### 【题目】

利用print()函数，打印出自己的学习目标和学习计划，以及对应的奖惩措施，即“flag学习法”

### 【提示】

“flag学习法”：规定时间——>目标——>惩罚和奖励措施。建议完成代码点击右上角分享到社交平台：如微信/朋友圈/QQ空间，让更多朋友见证你的进步。

### 【参考】

```
1 print('目标：用一个月的时间入门python，写出一个简单的项目!')
2
3 print('计划：每天一个小时学习Pyhton小课；每周复盘知识点；边学边练!')
4
5 print('惩罚：做不到，就给朋友圈所有点赞的亲友，每人发个一元红包!')
6
7 print('奖励：完成目标，奖励自己一款索尼蓝牙耳机!')
```

## 练习-听见无脸男的声音-参考

### 题目要求：

在千寻进入魔法世界的期间，有一位“无脸男”一直跟在她的身边。一开始，千寻对无脸男的跟踪感到害怕，直到她第一次听见了无脸男对她说的话后，才理解了他，并和他成为了朋友。

### 【讲解】

下面是无脸男对千寻说的话，请你把这段话打印出来，并且使用\*\*转义字符\*\*，把无脸男说的话按照正确格式打印：

千寻你好，人们叫我'无脸男'  
这个世界的人都选择无视我  
只有你看到了我并和我打招呼  
我感到很孤单，很孤单  
你愿意和我成为朋友吗？

### 题目讲解

1.复制下面这段无脸男对千寻说的话：

千寻你好，人们叫我'无脸男' 这个世界的人都选择无视我 只有你看到了我并和我打招呼 我感到很孤单，很孤单 你愿意和我成为朋友吗？

2.提示：文本中有引号的地方需要用到转义字符 \ ' ；需要换行的地方用 \n

### 【参考】

```
1 print("千寻你好，人们叫我'无脸男'\n这个世界的人都选择无视我\n只有你看到了我并和我打\n招呼\n我感到很孤单，很孤单\n你愿意和我成为朋友吗？")
```

## 第1关

### 练习-大话西游-参考

#### 【题目】

下面，是一串程序的运行结果。请根据运行结果来反推出它的代码应该怎么写。运行成功后，请你给代码加注释：

输入一个你错过的人：紫霞仙子

输入你最想对ta说的话：我爱你

输入一个时长(比如三分钟)：一万年

曾经有一份真挚的爱情放在我面前，那个人的名字是紫霞仙子

可我没有珍惜，如果上天能再给我一次机会，我会对ta说一句话：我爱你

如果非要给这句话加上一个期限，我希望是一万年

#### 【提示】

1. 利用input()函数获取用户输入的数据，并把这些数据放在一个变量中

示例：A = input('输入一个你错过的人：')

2. 在print()函数中，利用加号+实现字符串的拼接<br>

示例：print('曾经有一份真挚的爱情放在我面前，ta的名字是'+ A)

3. 代码添加注释，操作：#(文本内容)

#### 【示例】

A = input('输入一个你错过的人：') #用input()函数获取用户数据，并放到变量A里面

print('曾经有一份真挚的爱情放在我面前，ta的名字是'+A)

#通过加号，把获取到的数据A和字符串拼接起来，并用print()函数打印出来

运行结果：(输入 紫霞仙子)

输入一个你错过的人：紫霞仙子

曾经有一份真挚的爱情放在我面前，ta的名字是紫霞仙子

#### 【参考】

```
1 A = input('输入一个你错过的人：')      #用input()函数获取用户数据，并放到变量A里面
2 B = input('输入你最想对ta说的话：')    #用input()函数获取用户数据，并放到变量B里面
3 C = input('输入一个时长(比如三分钟)：')#用input()函数获取用户数据，并放到变量C里
4 面
5 print('曾经有一份真挚的爱情放在我面前，ta的名字是' + A)
6 #通过加号，把获取到的数据A和字符串拼接起来，并用print()函数打印出来
7 print('可我没有珍惜，如果上天能再给我一次机会，我会对他说三个字'+ B)
8 #通过加号，把获取到的数据B和字符串拼接起来，并用print()函数打印出来
9 print('如果非要给这句话加上一个期限，我希望是'+ C)
10 #通过加号，把获取到的数据C和字符串拼接起来，并用print()函数打印出来
```

### 练习-程序员的一人饮酒醉-参考

#### 题目要求

在Python的魔法世界，最常用的数据类型有三种：字符串(str)、整数(int)和浮点数(float)。

在\*\*数据拼接\*\*中，为了将不同的信息进行整合，可以使用\*\*拼接符号\*\*。但是，如果数据非字符串类型，则无法进行拼接。

此时，我们可以使用数据转换函数\*\*str()\*\*，将数据转换为字符串类型后，再进行拼接。

#### 【讲解】

请运用下方变量，使用str()函数打印两句话。

第一句话：1人我编程累碎掉的节操满地堆

第二句话：2眼是bug相随我只求今日能早归

可使用变量：

number1 = 1, number2 = 2

unit1 = '人', unit2 = '眼'

line1 = '我编程累', line2 = '是bug相随'

sentence1 = '碎掉的节操满地堆', sentence2 = '我只求今日能早归'

#### 题目讲解

1.将非字符串型变量转换为字符串类型

2.将字符串类变量拼接在一起

#### 【讲解】

1.使用str()函数

2.使用拼接符号\*\*+\*\*

3.使用print()函数

#### 【参考】

```
1 number1 = 1
2 number2 = 2
3 unit1 = '人'
4 unit2 = '眼'
5 line1 = '我编程累'
6 line2 = '是bug相随'
7 sentence1 = '碎掉的节操满地堆'
8 sentence2 = '我只求今日能早归'
9 first = str(number1) + unit1 + line1 + sentence1
10 second = str(number2) + unit2 + line2 + sentence2
11 print(first)
12 print(second)
```

#### 练习-非酋的吐槽-参考

##### 题目要求

在\*\*数据拼接\*\*中，为了将不同的信息进行整合，可以使用\*\*拼接符号\*\*。但是，如果数据非字符串类型，则无法进行拼接。此时，我们可以使用数据转换函数\*\*str()\*\*，将数据转换为字符串类型后，再进行拼接。但是有的时候，针对不同类型的数据，我们需要经历多次转换，最后才能转为字符串类型。

#### 【讲解】

请使用\*\*转换函数\*\*str()、int()、float()，运用下方变量，使用数据转换及数据拼接，输出结果： 脸黑怪我咯7张蓝票一个SSR都没有

可使用变量：

```
slogan = '脸黑怪我咯', number = '7.8', unit = '张', sentence = '蓝票一个SSR都没有'
```

### 题目讲解

- 1.由于变量中与小数形式的字符串，所以需要使用转换函数将此变量转换为浮点型
- 2.转换后使用int()函数取整
- 3.由于数据拼接需要变量为字符串形式，所以需要将变量转化为字符串类型
- 4.最后数据拼接，得出结果

#### 【讲解】

- 使用float()函数将小数形式的字符串变量转换
- 使用int()函数取整
- 使用str()函数将数据转换为字符串形式
- 使用\*\*+\*\*进行数据拼接

#### 【参考】

```
1 slogan = '脸黑怪我咯'
2 number = '7.8'
3 unit = '张'
4 sentence = '蓝票一个SSR都没有'
5 word = slogan + str(int(float(number))) + sentence
6 print(word)
```

## 第2关

### 练习-红豆-参考

#### 【题目】

请让这首诗实现换行的功能

```
print('《红豆》王维 红豆生南国, 春来发几枝。 愿君多采撷, 此物最相思。')
```

换行后，终端的运行效果如下：

```
《红豆》王维
红豆生南国, 春来发几枝。
愿君多采撷, 此物最相思。
```

#### 【提示】

可以使用转义字符 `【\n】`

#### 【参考】

```
1 print('《红豆》王维\n红豆生南国, 春来发几枝。 \n愿君多采撷, 此物最相思。')
```

### 练习-手机啊手机-参考

#### 【题目】

请输入每天花在手机上的娱乐时间 a，再把时间 a 乘以7，再打印——“您一周花在手机上的娱乐时间总共是xx小时”。

#### 【提示】

1. 用input()函数获取用户数据
2. 使用int()函数把数据转换为整数，再乘以7
3. 使用str()函数把一周花在手机上的娱乐时间转换为字符串，用print() 函数输出。

#### 【参考】

```
1 a = input('请输入你每天花在手机上的娱乐时间：')
2 b = int(a) * 7
3 print('您一周花在手机上的娱乐时间总共是'+str(b)+'小时')
```

## 练习-偷回宝石-参考

### 题目要求

在灭霸打了一个响指，宇宙一半生物都灰飞烟灭后，剩下的复仇者联盟成员们在巨大的创伤之中，依旧没有放弃反击灭霸的机会，他们决定利用最后一次行动机会，去把灭霸手里的宝石偷回来。最终，他们因为实力相差太大，1颗宝石都没有偷回来。

#### 【讲解】

请你写出一段代码：

如果偷回的宝石数是4颗及以上，输出结果“获得了打败灭霸的力量，反杀稳了”

如果偷回的宝石数是1-3颗，输出结果“可以全员出动，殊死一搏”

如果偷回的宝石数是0颗，输出结果“没办法了，只能尝试呼叫惊奇队长”

注意：标点符号都为英文格式

### 题目讲解

1.为变量【赋值】，写出“一个宝石都没偷回来”的情况

2.用【if】写第一个条件：如果偷回的宝石数是4颗及以上，输出结果：获得了打败灭霸的力量，反杀稳了

3.用【elif】写第二个条件：如果偷回的宝石数是1-3颗，输出结果：可以全员出动，殊死一搏

4.用【else】写第三个条件：如果偷回的宝石数是0颗，输出结果：没办法了，只能尝试呼叫惊奇队长

#### 【参考】

```
1 #以下为答案示例(stonenumber=0/1/2/3/4/5/6都是对的)
2 stonenumber=5
3 #偷回宝石数量赋值
4 if stonenumber >= 4:
5     #条件：如果你拥有的宝石数量大于等于4个
6     print('获取打败灭霸的力量')
7     #结果：显示‘获取打败灭霸的力量’的结果
8 elif 1 < stonenumber <= 3:
9     #条件：如果想让宝石数量停留在4至5个
10    print('可以全员出动，殊死一搏')
11    #结果：显示‘可以全员出动，殊死一搏’的结果
12 else:
```

```
13     #条件：当赋值不满足if和elif条件时，执行else下的命令，宝石数量在3个以下
14     print('没办法了，只能尝试呼叫惊奇队长')
15     #结果：显示‘没办法了，只能尝试呼叫惊奇队长’的结果
```

## 练习-美国队长的工资-参考

### 题目要求

复仇者联盟的成员也是有工资的，然而，由于美国队长一直被冰封在北极，错过了多次调薪机会，所以美国队长的工资一直是每月80美元。光荣挺进史塔克穷人榜前三名，并获封“美元队长”称号。

### 【讲解】

请你写出一段代码，判断美国队长的工资水平，代码需要满足如下条件：

如果月工资小于等于500美元，显示“欢迎进入史塔克穷人帮前三名”

如果月工资在100–500美元之间，显示“请找弗瑞队长加薪”

如果月工资小于等于100美元，显示“恭喜您荣获“美元队长”称号！”

如果月工资在500–1000美元之间（含1000美元），打印“祝贺您至少可以温饱了。”

其他情况下，如果工资大于1000美元，打印“经济危机都难不倒您！”

如果工资在1000–20000美元（含20000美元）之间，打印“您快比钢铁侠有钱了！”

如果月工资大于20000美元，打印“您是不是来自于瓦坎达国？”

不管赋值改变后输出结果如何，都需固定打印“程序结束”

### 题目讲解

- 1.根据题目，我们知道这里涉及嵌套条件，所以可以用扒洋葱法梳理代码逻辑
- 2.根据要求，先写最外层的\*\*if .....elif .....else.....\*\*条件
- 3.根据题目的从属关系分析，最外层的\*\*if\*\*条件和\*\*else\*\*条件都有额外条件
- 4.依次在外层基础条件下添加额外条件

### 【讲解】

- 1.使用if条件嵌套
- 2.嵌套条件需要缩进
- 3.注意单双引号同时存在时的互相转换
- 4.通过钱数输出对应的结果

### 【参考】

```
1  #以下为答案示例(salary为多少都是对的)
2  salary = 200
3  #对美国队长的工资赋值
4  if salary <= 500:
5      #条件：如果工资小于等于500
6      print('欢迎进入史塔克穷人帮前三名')
7      #结果：显示‘欢迎进入史塔克穷人帮前三名’的结果
8      if salary <= 100:
9          #条件：如果工资小于等于100
10         print('恭喜您荣获“美元队长”称号！')
11         #结果：显示‘恭喜您荣获“美元队长”称号！’的结果
12     else:
13         #条件：当条件满足第4行if，不满足第8行if，执行这个命令
```

```

14         print('请找弗瑞队长加薪')
15         #结果: 显示‘请找弗瑞队长加薪’的结果
16     elif 500 < salary <= 1000:
17         #条件: 如果工资大于500, 小于等于1000
18         print('祝贺您至少可以温饱了。')
19         #结果: 显示‘祝贺您至少可以温饱了。’的结果
20     elif salary > 1000:
21         #条件: 如果工资大于1000
22         print('经济危机都难不倒您! ')
23         #结果: 显示‘经济危机都难不倒您!’的结果
24         if 1000 < salary <= 20000:
25             #条件: 如果工资大于1000, 小于等于20000
26             print('您快比钢铁侠有钱了! ')
27             #结果: 显示‘您快比钢铁侠有钱了!’的结果
28         else:
29             #条件: 满足第20行elif条件, 不满足第24行if条件, 则执行这个命令
30             print('您是不是来自于瓦坎达国? ')
31             #结果: 显示‘您是不是来自于瓦坎达国?’的结果
32     print('程序结束')

```

## 第3关

### 练习-我要控制我自己-参考

#### 【背景】

最近, 派大星想要减肥, 他决定控制自己的饮食, 少吃一点蟹黄堡。海绵宝宝为了帮助好朋友派大星, 和派大星一起制定了一个饮食游戏。

游戏规则如下: 如果派大星一天吃超过40个蟹黄堡, 就要给海绵宝宝100块; 如果派大星一天吃小于等于40个的蟹黄堡, 海绵宝宝就给派大星100块。

#### 【题目】

请你写出一段代码: 当输入派大星一天吃的蟹黄堡数量时, 可以打印出这天该派大星给海绵宝宝钱, 还是海绵宝宝给派大星钱。

#### 【提示】

1. 使用input()函数获取获得派大星吃的蟹黄堡的数量。
2. 使用int() 函数强制类型转换, 将字符串转换为整数。
3. 使用if...else... 条件判断语句。

#### 【参考】

```

1 t = int(input('派大星今天吃了多少个蟹黄堡'))
2 if t > 40:
3     print('派大星给海绵宝宝100块')
4 else:
5     print('海绵宝宝给派大星100块')

```



## 练习-恋爱告白决策-参考

### 【题目】

使用Python代码来演示恋爱过程：

首先判断：两人的关系是不是达到了【朋友之上，恋人未满】？

如果不是，那么输出：进度条还不够，再等等。程序终结。

如果是，那么再判断：你是不是想和对方有进一步的发展？

如果不是，那么输出：还是做朋友吧。程序终结。

如果是，那么再判断：对方是不是想有进一步的发展？

如果不是，那么输出：恭喜获得“好人卡”。

如果是，那么输出：恭喜你们，有情人终成眷属！

### 【提示】

1. 条件判断的使用：if...elif...elif...else
2. 使用input()函数，不需要赋值也可以直接进行条件判断。比如：if input('吃了吗?')== '吃了':

### 【参考】

```
1 now = input('你们之前的感情有进入暧昧期了吗？回答有/没有：')
2
3 if now == '没有':
4     print('进度条还不太够，再等等吧。')
5 else:
6     p1 = input('你希望你们的关系再往前一步吗？回答想/不想：')
7     if p1 == '不想':
8         print('那还是做朋友吧。')
9     else:
10        p2 = input('那个人呢，ta想吗？回答想/不想：')
11        if p2 == '不想':
12            print('恭喜你！获得好人卡*1')
13        else:
14            print('恭喜你们！在一起！')
15
16 now = input('你们之前的感情有进入暧昧期了吗？回答有/没有：')
17 p1 = input('你希望你们的关系再往前一步吗？回答想/不想：')
18 p2 = input('那个人呢，ta想吗？回答想/不想：')
19
20 if now == '没有':
21     print('进度条还不太够，再等等吧。')
22 elif p1 == '不想':
23     print('那还是做朋友吧。')
24 elif p2 == '不想':
25     print('恭喜你！获得好人卡*1')
26 else:
27     print('恭喜你们！在一起！')
28
29 #提示是为了让大家更好写出代码，其实更优雅和符合逻辑的代码是这样的：
30
31 if input('你们之前的感情有进入暧昧期了吗？回答有/没有：') == '没有':
32     print('进度条还不太够，再等等吧。')
```

```

33 elif input('你们之前的感情有进入暧昧期了吗? 回答有/没有: ')=='没有':
34     print('那还是做朋友吧。')
35 elif input('那个人呢, ta想吗? 回答想/不想: ')=='不想':
36     print('恭喜你! 获得好人卡*1')
37 else:
38     print('恭喜你们! 在一起! ')
39
40 #我们是可以在写出代码后再进行优化的。
41
42 now = input('你们之前的感情有进入暧昧期了吗? 回答有/没有: ')
43 if now == '没有':
44     exit('进度条还不太够, 再等等吧。')
45 p1 = input('你希望你们的关系再往前一步吗? 回答想/不想: ')
46 if p1=='不想':
47     exit('那还是做朋友吧。')
48 p2 = input('那个人呢, ta想吗? 回答想/不想: ')
49 if p2=='不想':
50     exit('恭喜你! 获得好人卡*1')
51 else:
52     print('恭喜你们! 在一起! ')
53 #第三种答案
54
55 now = input('你们之前的感情有进入暧昧期了吗? 回答有/没有: ')
56
57 if now == '没有':
58     print('进度条还不太够, 再等等吧。')
59 else:
60     p1 = input('你希望你们的关系再往前一步吗? 回答想/不想: ')
61     if p1=='不想':
62         print('那还是做朋友吧。')
63     else:
64         p2 = input('那个人呢, ta想吗? 回答想/不想: ')
65         if p2=='不想':
66             print('恭喜你! 获得好人卡*1')
67         else:
68             print('恭喜你们! 在一起! ')
69 #第四种答案

```

## 练习-测测你健康吗? -参考

### 【题目】

输入你的体重和身高, 根据BMI指数的计算公式, 输出不同BMI指数对应的身体状况 (过轻、正常、过重、健康)。

### 【背景】

BMI指数是一个国际上常用的衡量人体胖瘦程度, 以及是否健康的指数。

它的计算方式如下:  $BMI = \text{体重} \div (\text{身高} \times \text{身高})$  比如一个体重50kg, 身高160cm的人,  $BMI = 50 \div (1.6 \times 1.6) = 19.5$

对照BMI的表格：

|    |               |
|----|---------------|
| 过轻 | BMI<18.5      |
| 正常 | 18.5≤BMI<24.0 |
| 过重 | 24.0≤BMI<28.0 |
| 肥胖 | 28.0≤BMI      |

by Python小课

### 【提示】

1. 使用条件判断语句if...elif...else的语句来写代码。
2. 使用input()函数获取输入的身高和体重数据。
3. 使用float()函数将字符串数据转换为浮点数。

### 【参考】

```
1 a = float(input('请输入你的体重(kg): '))
2 b = float(input('请输入你的身高(m): '))
3 bmi = a/(b*b)
4 if bmi >= 28.0:
5     print('肥胖')
6 elif bmi >= 24.0:
7     print('过重')
8 elif bmi >= 18.5:
9     print('正常')
10 else:
11     print('过轻')
12
13 #这并不是唯一的答案。
```

## 练习-哈利波特的宠物-参考

### 题目要求

哈利·波特在对角巷购买宠物时犯难了，他不知道该给自己的猫头鹰起什么名字。现在请你帮助哈利·波特，为他的猫头鹰起一个名字。

### 【讲解】

请你写一段代码：

为哈利·波特的猫头鹰起个名字，并打印出结果【哈利·波特的猫头鹰叫做XX】（XX是你起的名字）。

## 题目讲解

1.搜集猫头鹰的名字（知识点：input(),赋值）

2.打印结果【哈利·波特的猫头鹰叫做XX】（XX是你取的名字）（知识点：数据拼接）

### 【参考】

```
1 name = input('为哈利·波特的猫头鹰起个名字：')
2 print('哈利·波特的猫头鹰叫做' + name)
```

## 练习2-罗恩的减肥计划-参考

### 题目要求

为了三强争霸赛的舞会，罗恩想要减肥，所以他决定控制自己的饮食，少吃一点巧克力蛙。

哈利为了帮助好朋友罗恩，和他一起制定了一个饮食游戏。

游戏规则如下：

如果罗恩一天吃超过10个巧克力蛙，罗恩要给哈利100块；

如果罗恩一天吃小于等于10个的巧克力蛙，哈利就给罗恩100块。

### 【讲解】

请你写出一段代码：

当随机输入罗恩一天吃的巧克力蛙数量时，可以判断出这天，是罗恩给哈利钱，还是哈利给罗恩钱，并打印出来。

### 题目讲解

1.搜集罗恩吃的巧克力数量（知识点：input()、数据转换）

2.写第一个条件：如果罗恩一天吃超过10个巧克力蛙，罗恩要给哈利100块；

3.写第二个条件：如果罗恩一天吃小于等于10个的巧克力蛙，哈利就给罗恩100块。  
(知识点：if...else...)

### 【参考】

```
1 number = int(input('请输入罗恩吃的巧克力数量：'))
2 #int将其转换为整数，之后好进行大小的比较
3 if number > 10:
4     print('罗恩要给哈利100块')
5 else:
6     print('哈利就给罗恩100块')
7
```

## 练习3-古灵阁金币兑换-参考

### 题目要求

接受了霍格沃茨来信的你，带着欣喜与好奇，跟着魔咒学老师吴枫教授，来到了巫师世界的对角巷。在这个相当于人类集市的对角巷，你立刻被魔法世界稀奇古怪的东西吸引，想掏手机刷花呗买买买。但是吴枫教授说，麻瓜（没有魔法的人）货币在魔法世界不流通，但是你可以去

巫师世界的银行——古灵阁兑换货币。你立刻跟着吴枫老师跑到了古灵阁。当你进到巫师世界的银行时，就有银行的小精灵职员问好：

**\*\*注意：以下背景与题目要求相关\*\***

小精灵：您好，欢迎古灵阁，请问您需要帮助吗？需要or不需要？

你：需要

小精灵：请问您需要什么帮助呢？1 存取款；2 货币兑换；3 咨询

你：2

小精灵：金加隆和人民币的兑换率为1:51.3，即一金加隆=51.3人民币

小精灵：请问您需要兑换多少金加隆呢？

（你说了一个数字）

小精灵：好的，我知道了，您需要兑换（你刚说的数字）金加隆。

小精灵：那么，您需要付给我（将你刚说的数字转换为金加隆的数字）人民币。

注1：如果选择不需要帮助，小精灵会礼貌地说‘好的，再见。’

注2：如果选择帮助【1 存取款】，小精灵会推荐你去存取款窗口；如果选择帮助【3 咨询】，小精灵会推荐你去咨询窗口。

## 题目讲解

和小精灵的对话可以被这么拆分：

- 1.最基础的条件判断：需要帮助；不需要帮助（if...else...）
- 2.在需要帮助的情况下，又分为1存取款；2货币兑换；3咨询；三种情况。（if...elif...else...）
- 3.在2-货币兑换的情况下，需要再提一个问题，收集答案，打印出小精灵回答的结果（input()、数据拼接、数据转换）。

## 【参考】

```
1 chioce = input('您好，欢迎古灵阁，请问需要帮助吗？需要or不需要？')
2 if chioce == '需要':
3     number = input('请问您需要什么帮助呢？1 存取款；2 货币兑换；3 咨询')
4     if number == '2':
5         print('金加隆和人民币的兑换率为1:51.3，即一金加隆=51.3人民币')
6         print('请问您需要兑换多少金加隆呢？')
7         money = input('请输入你需要兑换的金加隆')
8         print('好的，我知道了，您需要兑换' + money + '金加隆。')
9         print('那么，您需要付给我'+str(int(money)*51.3)+'人民币。')
10
11     elif number == '1':
12         print('请到存取款窗口办理')
13
14     elif number == '3':
15         print('请到咨询窗口咨询')
16
17     else:
18         print('输入错误，没有你需要的服务')
19
20 elif chioce == '不需要':
21     print('好的，再见')
22
23 else:
24     print('输入错误')
```

## 第4关

### 练习-君子爱‘数’取之有道-参考

#### 题目讲解

第一步：取出列表中的第三个元素（list1[2]），字典{'爱':'love'}

第二步：取出list1[2]中键'爱'所对应的值，即'love'（list1[2]['爱']）

#### 【参考】

```
1 list1 = [{'嫉妒':'envy'},{'恨':'hatred'},{'爱':'love'}]
2 print(list1[2]['爱'])
```

#### 题目讲解

第一步：取出字典中键为3对应的值（dict1[3]），即['love','hatred','envy']

第二步：再取出列表['love','hatred','envy']中的第一个元素（dict1[3][0]）

#### 【参考】

```
1 dict1 = {1:['cake','scone','puff'],2:['London','Bristol','Bath'],3:
  ['love','hatred','envy']}
2 print(dict1[3][0])
```

#### 题目讲解

第一步：元组和列表都是序列,提取的方式也是偏移量，如 tuple1[1]、tuple1[1:]。

第二步：先取出列表中的第二个元素中的（list2[1]）

第三步：然后在第二个元素的基础上在去取值，即list2[1][1]

注：（元组和列表的取值方式是相同的）

#### 【参考】

```
1 tuple1 = ('A','B')
2 list2 = [('A','B'),('C','D'),('E','F')]
3
4 print(tuple1[0])
5 print(list2[1][1])
```

### 练习-找到那只狼-参考

#### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。<br>

练习目标：

在层层嵌套的各种数据类型中，准确地提取出你需要的数据。<br>

#### 练习要求：

在未来世界里，一个新建的童话镇吸引了不少人入住。

不过，在人群里隐藏着一只狼，会威胁大家的安全。

童话镇的镇长希望你能找到它，并揭发其身份。

用程序语言就是说：列表中有个字符串是“狼”，将其打印出来吧。

#### 【解答】

```
1 townee = [  
2     {'海底王国':['小美人鱼','海之王','小美人鱼的祖母','五位姐姐'],'上层世界':['王  
   子','邻国公主']},  
3     '丑小鸭','坚定的锡兵','睡美人','青蛙王子',  
4     [{'主角':'小红帽','配角1':'外婆','配角2':'猎人'},{'反面角色':'狼'}]  
5 ]  
6  
7 print(townee[5][1]['反面角色'])  
8
```

## 第5关

### 练习-数数字-参考

#### 题目讲解

小美想知道你是怎么用 for循环 的，在下方写下你的代码吧~

小美还想知道你是怎么用 while循环 的，在下方写下你的代码吧~

#### 【参考】

```
1 # while 循环  
2 n = 0  
3 while n < 7:  
4     n = n+1  
5     if n != 4: # 当num != 4，执行打印语句；等于4时不打印。  
6         print(n)
```

```
1 # for 循环  
2 for num in range(1,8): # 为同时能运行两个循环，新取参数 num。  
3     if num != 4: # 当num != 4，执行打印语句；等于4时不打印。  
4         print(num)
```

### 练习-轮流坐前排-参考

#### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 练习目标：

通过这个练习，你会尝试用循环来解决生活中的问题，并了解一种新的列表方法。

### 练习要求：

小明、小红、小刚是同班同学，且坐在同一排，分别坐在第一位、第二位、第三位。

由于他们的身高都差不多，所以，老师计划让他们三个轮流坐在第一位。

每次换座位的时候，第一位变第三位，后面两位都往前一位。

方法1：append()函数

### 【提示】

这里需要用到列表的切片和索引。

### 【解答】

```
1 students = ['小明', '小红', '小刚']
2 for i in range(3):
3     student1 = students[0]
4     students = students[1:]
5     students.append(student1)
6     print(students)
```

参考代码中用的是 for 循环，当然，while 循环 也是可以的。

感兴趣的同学，可以两者都试试，然后对比一二。

### 方法2：pop()函数

我们先介绍一下列表中的pop()函数，用于移除列表中的一个元素（默认最后一个元素），并且返回该元素的值。

可以将其理解为提取和删除的融合：①提取：取到元素，对列表没有影响；②删除：删除列表的元素。

而移除，则是同时做到取到元素，并且删除列表中的元素。

### 【提示】

你可以先运行一下右侧的头几行代码感受一下，然后再运用pop()函数来满足“轮流坐前排”这个需求。

### 【解答】

```
1 students = ['小明', '小红', '小刚']
2 for i in range(3):
3     student1 = students.pop(0)
4     students.append(student1)
5     print(students)
6
```

相信你已经用了新学的方法完成了代码。可以发现：代码从6行减为5行，虽然仅仅1行代码的差别，还不能很好地体现“简化”代码的思想，但这是一种必备的编程思维。

当你发现要实现某功能要用较多代码实现，最先做的不是埋头打代码，而是找找是否有更好、更简洁的解决方法。



## 第6关

### 练习-囚徒困境-参考

#### 【讲解】

1.开启循环，两人分别选择

首先，我们需要知道两个囚徒各自的选择（input\*2）。

2.循环当中，有判断和跳出

两个囚徒的选择，会有四种不同的结果。所以，可以用'if...elif...else'。

#### 判决提示：

若两人都认罪，则两人各判10年；

若一个认罪一个抵赖，则认罪的人判1年，抵赖的人判20年；

若两人都抵赖，则各判3年——这种情况下跳出循环。

#### 【参考】

```
1 while True:
2     a = input('A, 你认罪吗？请回答认罪或者不认')
3     b = input('B, 你认罪吗？请回答认罪或者不认')
```

```
1 while True:
2     a = input('A, 你认罪吗？请回答认罪或者不认')
3     b = input('B, 你认罪吗？请回答认罪或者不认')
4     if a == '认罪' and b == '认罪':
5         print('两人都得判10年，唉')
6     elif a == '不认' and b == '认罪':
7         print('A判20年，B判1年，唉')
8     elif a == '认罪' and b == '不认':
9         print('A判1年，B判20年')
10    else:
11        print('都判3年，太棒了')
12        break # 当满足开头提到的条件时，跳出循环。
```

### 练习-记录困境中的选择-参考

#### 练习要求

上一个练习，我们将“囚徒困境”写成了代码，让程序收集两名囚犯的认罪情况，进而决定他们的判决：

两人都认罪，则各判10年；一个认罪一个抵赖，则前者判1年，后者判20年；两人都抵赖，各判3年。只有两人都都不认罪，程序才会停止。

现在有一个社会学家，在不同的人群中做这个实验，一旦遇到都不认罪的情况，就停止该人群中的实验。

同时，他希望程序能记录每一对实验者的选择，以及记录第几对实验者都选择不认罪。请你帮帮他吧。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 练习目标：

这个作业会建立在上一个练习的基础上，完善代码的功能。

### 几个提示：

为了记录每一对实验者的选择，需要用一个可拓展的“容器”来存放这些数据；

为了记录是第几对实验者做了最优选择，需要用一个变量来计数；

为了将每一对实验者的选择再打印出来，需要写一个循环。

在打印的那个循环里，需要留意列表是从0开始计数的。

### 【解答】

```
1  n = 0
2  list_answer = []
3
4
5  while True:
6      n += 1
7      a = input('A, 你认罪吗? 请回答认罪或者不认: ')
8      b = input('B, 你认罪吗? 请回答认罪或者不认: ')
9      list_answer.append([a,b])
10     if a == '认罪' and b == '认罪':
11         print('两人都得判10年, 唉')
12     elif a == '不认' and b == '认罪':
13         print('A判20年, B判1年, 唉')
14     elif a == '认罪' and b == '不认':
15         print('A判1年, B判20年')
16     else:
17         print('都判3年, 太棒了')
18         break
19
20
21 print('第' + str(n) + '对实验者选了最优解。')
22
23
24 for i in range(n):
25     print('第' + str(i+1) + '对实验者的选择是: ' + str(list_answer[i]))
26
27
```

## 练习-演员的作品-参考

### 练习要求

我很喜欢看电影，我回忆了一下，这两年我觉得还不错的国产电影。

下面，会将电影的影片名和主演放在字典里，如`movie = {'妖猫传':['黄轩','染谷将太']}`。

需要你补充一些代码，让其他人只要输入演员名，就打印出：xx出演了电影xx。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 练习目标：

通过这个练习，你会学会综合使用循环、数据提取和条件判断。

注：这个练习的提示比较完整，建议先多尝试几次独立完成。

### 【提示】

对该练习完整的提示：

- 1.先用for...in...遍历字典，把键（即电影名）取出来；
- 2.再用字典的键把字典里的值（即包含主演名字列表）取出来；
- 3.最后用if A in B 作为条件判断A在不在B中，即输入的演员名在不在这个列表中。如果在这个列表中，就打印出xx出演了电影xx。

### 【解答】

```
1 movies = {  
2     '妖猫传':['黄轩','染谷将太'],  
3     '无问西东':['章子怡','王力宏','祖峰'],  
4     '超时空同居':['雷佳音','佟丽娅'],  
5 }  
6  
7  
8 actor = input('你想查询哪个演员? ')  
9 for movie in movies:  
10     actors = movies[movie]  
11     if actor in actors:  
12         print(actor + '出演了电影' + movie)
```

## 第7关

### 练习-再来一盘-参考

### 【讲解】

要达成目标，有两种方案：

#### 1.while True+break:

开启一个无限循环，设定跳出条件。

当得到肯定回复时，继续运行；当得到否定回复时，运行break，停止循环，退出游戏。

## 2.while 变量名+变量名的布尔值判断：

在开头设某变量的布尔值为True，input后开启判断变量的布尔值是否改变。

当得到肯定回复时，while True，继续运行；当得到否定回复时，while False，停止循环，退出游戏。下面，我们先用较简单的代码来体会一下这两种方案。

### 方案1：while True+break

开启一个无限循环，设定跳出条件。

当得到肯定回复时，继续运行；当得到否定回复时，运行break，停止循环，退出游戏。

### 方案2：while 变量名+变量名的布尔值判断

在开头设某变量的布尔值为True，input后开启判断变量的布尔值是否改变。

当得到肯定回复时，while True，继续运行；当得到否定回复时，while False，停止循环，退出游戏。

```
1 # 方案1
2 while True:
3     a1 = input('要继续游戏吗，请输入n退出，输入其他继续：')
4     if a1 == 'n':
5         break
6
7 # 方案2
8 again = True
9 while again:
10    a2 = input('要继续游戏吗，请输入y继续，输入其他退出：')
11    if a2 == 'y':
12        again = True
13    else:
14        again = False
15
```

## 【参考】

```
1 #for循环
2 import time
3 import random
4
5 player_victory = 0
6 enemy_victory = 0
7
8 for i in range(1,4):
9     time.sleep(1.5)
10    print(' \n———现在是第 %s 局———' % i)
11    player_life = random.randint(100,150)
12    player_attack = random.randint(30,50)
13    enemy_life = random.randint(100,150)
14    enemy_attack = random.randint(30,50)
15
16    print('【玩家】\n血量: %s\n攻击: %s' % (player_life,player_attack))
```

```

17     print('-----')
18     time.sleep(1)
19     print('【敌人】\n血量: %s\n攻击: %s' % (enemy_life, enemy_attack))
20     print('-----')
21     time.sleep(1)
22
23     while player_life > 0 and enemy_life > 0:
24         player_life = player_life - enemy_attack
25         enemy_life = enemy_life - player_attack
26         print('你发起了攻击, 【玩家】剩余血量%s' % player_life)
27         print('敌人向你发起了攻击, 【敌人】的血量剩余%s' % enemy_life)
28         print('-----')
29         time.sleep(1.2)
30
31     if player_life > 0 and enemy_life <= 0:
32         player_victory += 1
33         print('敌人死翘翘了, 你赢了! ')
34     elif player_life <= 0 and enemy_life > 0:
35         enemy_victory += 1
36         print('悲催, 敌人把你干掉了! ')
37     else:
38         print('哎呀, 你和敌人同归于尽了! ')
39
40     if player_victory > enemy_victory :
41         time.sleep(1)
42         print('\n【最终结果: 你赢了! 】')
43     elif enemy_victory > player_victory:
44         print('\n【最终结果: 你输了! 】')
45     else:
46         print('\n【最终结果: 平局! 】')
47
48
49 #while 循环
50 import time
51 import random
52
53 player_victory = 0
54 enemy_victory = 0
55
56 while True:
57     for i in range(1,4):
58         time.sleep(1.5)
59         print(' \n—————现在是第 %s 局—————' % i)
60         player_life = random.randint(100,150)
61         player_attack = random.randint(30,50)
62         enemy_life = random.randint(100,150)
63         enemy_attack = random.randint(30,50)
64
65         print('【玩家】\n血量: %s\n攻击: %s' % (player_life, player_attack))
66         print('-----')
67         time.sleep(1)

```

```

68     print('【敌人】\n血量: %s\n攻击: %s' % (enemy_life, enemy_attack))
69     print('-----')
70     time.sleep(1)
71
72     while player_life > 0 and enemy_life > 0:
73         player_life = player_life - enemy_attack
74         enemy_life = enemy_life - player_attack
75         print('你发起了攻击, 【玩家】剩余血量%s' % player_life)
76         print('敌人向你发起了攻击, 【敌人】的血量剩余%s' % enemy_life)
77         print('-----')
78         time.sleep(1.2)
79
80     if player_life > 0 and enemy_life <= 0:
81         player_victory += 1
82         print('敌人死翘翘了, 你赢了! ')
83     elif player_life <= 0 and enemy_life > 0:
84         enemy_victory += 1
85         print('悲催, 敌人把你干掉了! ')
86     else:
87         print('哎呀, 你和敌人同归于尽了! ')
88
89     if player_victory > enemy_victory :
90         time.sleep(1)
91         print('\n【最终结果: 你赢了! 】')
92     elif enemy_victory > player_victory:
93         print('\n【最终结果: 你输了! 】')
94     else:
95         print('\n【最终结果: 平局! 】')
96
97     a1 = input('要继续游戏吗, 请输入n退出, 输入其他继续: ')
98     if a1 == 'n':
99         break

```

## 练习-再来一种-参考

### 练习目标

在这个练习, 我们会学会一种新的“格式化字符串”的方法: `format()`函数。

### 练习要求

在项目1的末尾, 我们学会了一种简化代码的方式: 格式化字符串。

不过, 还有一种更强大的方法, 下面我们会先学习, 然后再练习。

### 【讲解】

明确目标很重要 (所以重复三遍) 。

做到后面的步骤, 可再点开左侧的“练习介绍”查看。

## 学习`format()`函数

format()函数是从 Python2.6 起新增的一种格式化字符串的函数，功能比课堂上提到的方式更强大。

format()函数用来占位的是大括号{}，不用区分类型码（%+类型码）。

具体的语法是：'str.format()'，而不是课堂上提到的'str % ()'。

而且，它对后面数据的引用更灵活，不限次数，也可指定对应关系。

看完左侧的代码、结果和注释，你就懂上面几句话的意思了。

### 运用format()函数

将代码中字符串格式化的代码改成format()函数的方法，改完后运行一下，检验是否正确。

#### 【提示】

一个小提示：一共有5行代码需要改，找到它们，然后改下。

#### 【解答】

```
1  import time
2  import random
3
4
5  player_victory = 0
6  enemy_victory = 0
7
8
9  while True:
10     for i in range(1,4):
11         time.sleep(1.5)
12         print(' \n—————现在是第 {} 局—————'.format(i))
13         player_life = random.randint(100,150)
14         player_attack = random.randint(30,50)
15         enemy_life = random.randint(100,150)
16         enemy_attack = random.randint(30,50)
17
18
19         print('【玩家】\n血量: {}\n攻击:
{}'.format(player_life,player_attack))
20         print('-----')
21         time.sleep(1)
22         print('【敌人】\n血量: {}\n攻击:
{}'.format(enemy_life,enemy_attack))
23         print('-----')
24         time.sleep(1)
25
26
27     while player_life > 0 and enemy_life > 0:
28         player_life = player_life - enemy_attack
29         enemy_life = enemy_life - player_attack
30         print('敌人发起了攻击，【玩家】剩余血量{}'.format(player_life))
31         print('你发起了攻击，【敌人】的血量剩余{}'.format(enemy_life))
32         print('-----')
33         time.sleep(1.2)
34
```

```

35
36     if player_life > 0 and enemy_life <= 0:
37         player_victory += 1
38         print('敌人死翘翘了，你赢了! ')
39     elif player_life <= 0 and enemy_life > 0:
40         enemy_victory += 1
41         print('悲催，敌人把你干掉了! ')
42     else:
43         print('哎呀，你和敌人同归于尽了! ')
44
45
46     if player_victory > enemy_victory :
47         time.sleep(1)
48         print('\n【最终结果：你赢了! 】')
49     elif enemy_victory > player_victory:
50         print('\n【最终结果：你输了! 】')
51     else:
52         print('\n【最终结果：平局! 】')
53
54
55     a1 = input('要继续游戏吗，请输入n退出，输入其他继续: ')
56     if a1 == 'n':
57         break
58
59

```

## 第8关

### 练习-老师我帮你-参考

#### 1. 合并列表-1

分析问题，明确结果

我们的问题是：将两个列表合并起来，并按照从低到高的顺序排序，要得到问题的结果，我们还需要借助打印函数看看我们合并的是否正确。

#### 思考要用到的知识&思考切入点

增加列表的内容与合并列表的含义相同，所以我们可以使用append作为解题的切入点，请你试试!

**提示：**假设要用已学过的知识来完成这道作业，需要用到列表的新增函数和循环。

```

1 list1 = [91, 95, 97, 99]
2 list2 = [92, 93, 96, 98]
3 for i in list2:
4     list1.append(i)
5 print(list1)

```



## 2.合并列表-2

上网搜索新知识&找到新的切入点

好了。你已经完成了第一个需求：合并列表。不过，有没有发现，这个代码还是比较冗余的。

有没有更简单的方法呢？请你自己上网搜索一下

`python 合并两个列表`，看看是否有更简单的方法（提示：以字母e开头），学会后再回来继续做作业吧。

请你根据新学到的知识，简化代码。

注（如果你直接将 list2 合并到 list1 上，那就无法做到只看A组的成绩，所以，最好再建一个列表来合并两组的成绩）

```
1 list1 = [91, 95, 97, 99]
2 list2 = [92, 93, 96, 98]
3 # 把 A 组成绩赋值给一个新列表，用来存合并的成绩——这个细节要注意！
4 list3 = list1
5 list3.extend(list2)
6 print(list3)
```

## 3.列表排序

主动搜索，掌握新知

老师鼓励你自己去探寻解决问题的方法，上网搜索一下列表的排序方法吧，是不是发现了一个以字母s开头的方法？

```
1 list3= [91, 95, 97, 99, 92, 93, 96, 98]
2 list3.sort()
3 print(list3)
```

## 练习-老师我又来了-参考

### 练习目标

这个练习，是建立在上一个练习之上，用代码帮老师完成更多的成绩处理工作。

### 练习要求

上一个练习中，我们完成了两组成绩的合并和排序。

不过，老师有了新的需求：想知道两组的平均分，以及把低于平均分的成绩也打印出来。

所以，在这个练习中，我们会帮老师计算出两组的平均分，并挑出那些在平均分之下的成绩。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 【提示】

解题三连击：

1.目前我们想要的结果是：求平均值和判断比较；

2.我们可以用到的知识有：循环叠加成绩，然后除以总人数，即可求平均值。判断比较用小于号和空列表即可；

3.我们的切入点：由于学生的成绩已经被我们集中到一个列表里，所以可以用for循环遍历这个列表来取出小于平均值的成绩。

经过三连击，相信你已经可以帮老师自动化地达成目标了。

#### 【解答】

```
1 scores1 = [91, 95, 97, 99, 92, 93, 96, 98]
2 sum = 0
3 scores2 = []
4
5
6 for score in scores1:
7     sum = sum + score
8     average = sum/len(scores1)
9 print('平均成绩是: {}'.format(average))
10
11
12 for score in scores1:
13     if score < average:
14         scores2.append(score)
15         continue
16 print(' 低于平均成绩的有: {}'.format(scores2))
17
18
```

#### 第二种 解题三连击：

1.目前我们想要的结果是：求平均值和判断比较；

2.我们可以去找的新知识有：Python 求平均值；

3.我们的切入点：请你通过搜索，找到更简单的求平均值的方法，来改造代码。

#### 【提示】

这个方法的单词首字母是“n”，性质是“拓展程序库”。在改造代码前，可以先读懂网上的案例。

#### 【解答】

```
1 import numpy as np
2
3
4 scores1 = [91, 95, 97, 99, 92, 93, 96, 98]
5 scores2 = []
6
7
8 average = np.mean(scores1) # 一行解决。
9 print('平均成绩是: {}'.format(average))
10
11
12 for score in scores1:
13     if score < average:
14         scores2.append(score)
15         continue
```

```
16 print(' 低于平均成绩的有: {}'.format(scores2))
17
18
```

## 第9关

### 练习-Hellokitty抽奖器-参考

练习目标：

我们会通过今天的项目练习，学习函数的封装和调用。

练习要求：

我们已经有一个hellokitty抽奖器，现在，请你把这个程序封装成一个新的函数。

#### 运行抽奖器的代码参考

```
1 import random
2 import time
3
4 # 用random函数在列表中随机抽奖，列表中只有3位候选者。
5 luckylist = ['海绵宝宝','派大星','章鱼哥']
6 # random模块中有个随机选取一个元素的方法：random.choice()。
7 a = random.choice(luckylist) # 从3个人中随机选取1个人。
8 print('开奖倒计时',3)
9 time.sleep(1) # 调用time模块，控制打印内容出现的时间
10 print('开奖倒计时',2)
11 time.sleep(1)
12 print('开奖倒计时',1)
13 time.sleep(1)
14 # 使用三引号打印hellokitty的头像
15 image = '''
16 /\_)o<
17 |      \\\
18 | 0 . 0|
19 \_____/
20 '''
21 print(image) # .....
22 print('恭喜'+a+'中奖! ') # 使用print函数打印幸运者名单
23 import random
24 import time
```

运行代码，熟悉了解抽奖器抽奖的原理，请仔细查看注释讲解。

#### 分析代码结构，完成函数封装

```
1 # 提示：将以下部分封装进函数
```

```

2 luckylist = ['海绵宝宝','派大星','章鱼哥']
3 a = random.choice(luckylist)
4 print('开奖倒计时',3)
5 time.sleep(1)
6 print('开奖倒计时',2)
7 time.sleep(1)
8 print('开奖倒计时',1)
9 time.sleep(1)
10 image = '''
11 /\_)o<
12 |      \\\
13 | 0 . 0|
14 \_____/
15 '''
16 print(image)
17 print('恭喜'+a+'中奖! ')
18 # 查看注释，运行代码。
19 import random
20 import time

```

### 抽奖程序分为两部分：

我们需要将第二部分用函数封装起来，并调用函数。提示：3个参与者即函数有3个参数。

```

1 import random
2 import time
3
4 # 将抽奖程序封装成函数
5 def choujiang(q,w,e): # 定义一个抽奖函数，带有3个参数，也就是3位候选人
6     luckylist = [q,w,e] # 定义一个中奖名单的列表
7     a = random.choice(luckylist) # 在中奖名单里面随机选择
8     print('开奖倒计时',3)
9     time.sleep(1)
10    print('开奖倒计时',2)
11    time.sleep(1)
12    print('开奖倒计时',1)
13    time.sleep(1)
14    image = '''
15    /\_)o<
16    |      \\\
17    | 0 . 0|
18    \_____/
19    '''
20    print(image)
21    print('恭喜'+a+'中奖! ')
22    choujiang('虚竹','萧峰','段誉') # 调用函数

```

### 练习-生成一副扑克牌-参考

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 练习目标：

通过这个练习，你可以通过Python优雅地生成一副扑克牌。

### 练习要求：

我们将通过这个练习，简单地复习一下return的用法。

另外，这个练习也要求你能够快速地学习新知识并将其运用出来。

新知识有3个：一种新的列表生成方式、extend 的新用法和列表生成式。

### 三个新知识

请仔细查看右侧的代码和注释。重要的几行代码：

```
1 num2 = list(range(1,6))
2
3 num2.extend('ABCDE')
4
5 list2 = [m+n for m in ['天字', '地字'] for n in '一二']
6
7 list3 = [n*n for n in range(1,11) if n % 3 == 0]
```

请根据学到的新知识，补全函数cards()。

每张扑克牌的展现形式是一个元组：(花色，大小)。

### 【解答】

```
1 def cards():
2     color = ['红心', '方块', '梅花', '黑桃']
3     num = list(range(2, 11))
4     num.extend('JQKA')
5     return [(x, y) for x in color for y in num ]
6
7
8 print(cards())
9
10
```

## 第10关

### 练习-剪刀石头布-参考

### 练习要求：

和电脑玩一个剪刀石头布的游戏：电脑随机出拳，我们可选择出什么。

### 练习目标：

我们会通过今天的作业，做出和电脑进行“石头剪刀布”的游戏。

### 练习要求：

和电脑玩一个剪刀石头布的游戏：电脑随机出拳，我们可选择出什么。

#### 1.双方出拳

首先，我们要让双方选择出拳，才能判断胜负。

我们可以设置变量computer\_choice代表电脑的出拳选择，设置变量user\_choice代表你的出拳选择。

电脑的出拳，我们可以使用random.choice()来随机选择；我们的出拳，可以手动输入我们出拳的类型。

另外，判断下输入：当输入的内容不是石头剪刀布时，电脑会提醒‘输入有误，请重新出拳’，并重新出拳。

```
1 import random
2 # 出拳
3
4 punches = ['石头','剪刀','布']
5 computer_choice = random.choice(punches)
6 user_choice = ''
7 user_choice = input('请出拳：（石头、剪刀、布）') # 请用户输入选择
8 while user_choice not in punches: # 当用户输入错误，提示错误，重新输入
9     print('输入有误，请重新出拳')
10    user_choice = input()
11
```

#### 2.双方亮拳

你和电脑已经对自己要出的拳进行了选择，接下来，我们需要知道双方的出拳类型。

请使用print()函数补充亮拳的结果。

```
1 import random
2
3 # 出拳
4 punches = ['石头','剪刀','布']
5 computer_choice = random.choice(punches)
6 user_choice = ''
7 user_choice = input('请出拳：（石头、剪刀、布）') # 请用户输入选择
8 while user_choice not in punches: # 当用户输入错误，提示错误，重新输入
9     print('输入有误，请重新出拳')
10    user_choice = input()
11
12 # 亮拳
13 print('——战斗过程——')
14 print('电脑出了： %s' %(computer_choice))
15 print('你出了： %s' %(user_choice))
16
```

#### 3.判断胜负

在前面两步，电脑和你已经选择完出拳的类型并亮拳后，只差最后一步：根据结果判断胜负。  
胜负分为三种情况：  
两方出拳一致：平局；  
你出石头，电脑出剪刀；你出剪刀，电脑出布；你出布，电脑出石头：你获胜；  
其他情况：你输了。  
根据上面的逻辑将代码补充完整，判断胜负，试试能否运行。

```
1 import random
2
3 # 出拳
4 punches = ['石头','剪刀','布']
5 computer_choice = random.choice(punches)
6 user_choice = ''
7 user_choice = input('请出拳：（石头、剪刀、布）') # 请用户输入选择
8 while user_choice not in punches: # 当用户输入错误，提示错误，重新输入
9     print('输入有误，请重新出拳')
10    user_choice = input()
11
12 # 亮拳
13 print('——战斗过程——')
14 print('电脑出了：%s' %(computer_choice))
15 print('你出了：%s' %(user_choice))
16
17 # 胜负
18 print('——结果——')
```

### 参考答案

```
1 punches = ['石头','剪刀','布']
2 computer_choice = random.choice(punches)
3 user_choice = ''
4 user_choice = input('请出拳：（石头、剪刀、布）') # 请用户输入选择
5 while user_choice not in punches: # 当用户输入错误，提示错误，重新输入
6     print('输入有误，请重新出拳')
7     user_choice = input()
8
9 # 亮拳
10 print('——战斗过程——')
11 print('电脑出了：%s' %(computer_choice))
12 print('你出了：%s' %(user_choice))
13
14 # 胜负
15 print('——结果——')
16 if user_choice == computer_choice: # 使用if进行条件判断
17     print('平局！')
18 elif (user_choice == '石头' and computer_choice == '剪刀') or (user_choice
19 == '剪刀' and computer_choice == '布') or (user_choice == '布' and
20 computer_choice == '石头'):
21     print('你赢了！')
22 else:
23     print('你输了！')
```

## 练习-让代码更简洁-参考

### 练习目标

我们会通过这个练习，简化上一个练习“石头剪刀布”的代码。

### 练习要求

上一个练习的代码中，有一个判断语句的代码很长很长：

```
1 elif (user_choice == '石头' and computer_choice == '剪刀') or (user_choice == '剪刀' and computer_choice == '布') or (user_choice == '布' and computer_choice == '石头'):
```

我们会通过一个新的知识，将其简化，体验到“知识得增加，代码得简化”这个客观规律。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### index()函数

index() 函数用于找出列表中某个元素第一次出现的索引位置。

语法为：list.index(obj)，obj为object（对象）的缩写。

具体可参考右侧的代码和运行结果。

现在，请你根据新学的函数去简化代码吧。

### 【提示】

提示：可以先确定电脑随机选到的选项在列表punches中的索引位置，将其和玩家的选项联系起来。

### 【解答】

```
1 import random
2
3
4 punches = ['石头','剪刀','布']
5 computer_choice = random.choice(punches)
6 user_choice = ''
7 user_choice = input('请出拳：（石头、剪刀、布）')
8 while user_choice not in punches:
9     print('输入有误，请重新出拳')
10    user_choice = input()
11
12
13 print('——战斗过程——')
14 print('电脑出了： %s' % computer_choice)
15 print('你出了： %s' % user_choice)
16
17
18 print('——结果——')
```



```

19 if user_choice == computer_choice:
20     print('平局! ')
21 elif user_choice == punches[punches.index(computer_choice)-1]:
22     print('你赢了! ')
23 else:
24     print('你输了! ')
25
26

```

## 第11关

### 练习—一起来捉虫—参考

#### 1.知识点debug

##### 【参考】

```

1 scores = {'语文':89, '数学':95, '英语':80}
2 def get_average(scores):
3     sum_score = 0
4     for subject, score in scores.items():
5         sum_score += score
6         print('现在的总分是%d'%sum_score)
7     ave_score = sum_score/len(scores)
8     print('平均分是%d'%ave_score)
9
10
11 get_average(scores)

```

#### 2.思维不清debug

##### 【参考】

```

1 not_bad_word = True
2 while not_bad_word:
3     x = input('请给旺财取个外号: ')
4     if x == '小狗' or x == '汪汪': # 只要外号是两个中的一个, 就会生气。
5         not_bad_word = False
6         print('我生气了, 不想理你了! ')
7
8
9 print('对不起, 以后我不会这么叫你了')

```

#### 3.被动掉坑debug

##### 【参考】

```

1 deposit = [100,300,900,2000,5000,0,2000,4500]
2
3 for i in range(1, len(deposit)):
4     if deposit[i-1] == 0: # 判断被除数等于0时, 特殊处理。

```

```
5     print('你上次存款为 0 哦! ')
6     else:
7         times = deposit[i]/deposit[i-1]
8         print('你的存款涨了%f倍'%times)
```

## 练习-贴心的除法计算器-参考

### 练习目标：

通过这个练习，我们会用代码做出一个贴心的除法计算器：只要输入有误，就会给出相应的报错信息。

### 练习要求：

这个除法计算器需要包含的报错信息有：输入了非数值（即不属于整数和浮点数）、被除数为零以及变量不存在。

为了让代码可以给出相应的报错信息，我们可以运用课堂中谈到的`try...except`语句。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 回顾和升级

我们以课堂中的一个代码为例，回顾并升级对`try...except`语句的用法。

请你按照右侧代码区的提醒阅读和运行代码，了解带多个except的try结构。

### 【提示】

有兴趣的话，可阅读课堂末的链接，了解`try...except`的更多用法和变形。

请你找到右侧的代码可能会出现的几种报错（可运行尝试），然后用带多个except的try结构来解决这些可能出现的输入错误。

第二个报错信息是：ValueError，你找到了吗？

### 【解答】

```
1 print('\n欢迎使用除法计算器！\n')
2
3
4 while True:
5     try:
6         x = input('请输入被除数：')
7         y = input('请输入除数：')
8         z = float(x)/float(y)
9     except ZeroDivisionError:
10        print('0是不能做除数的！')
11    except ValueError:
12        print('除数和被除数都应该是整值或浮点数！')
13    else:
14        print(x,'/',y,'=',z)
15        break
```

16  
17

## 第12关

### 练习-员工管理系统-参考

#### 1.新增业绩的类方法

##### 【参考】

```
1  # 创建一个人事系统类
2  class hrSystem:
3      # 创建存储员工名字的变量 name
4      name = ''
5      # 创建存储员工工资的变量 salary
6      salary = 0
7      # 创建存储员工绩效的变量 kpi
8      kpi = 0
9
10     # 定义录入员工信息的类方法
11     @classmethod
12     def record(cls, name, salary, kpi):
13         cls.name = name
14         cls.salary = salary
15         cls.kpi = kpi
16
17     # 定义打印员工信息的类方法
18     @classmethod
19     def print_record(cls):
20         print(cls.name + '的工作信息如下: ')
21         print('本月工资: ' + str(cls.salary))
22         print('本年绩效: ' + str(cls.kpi))
23
24     # 定义根据 kpi 评奖的类方法
25     @classmethod
26     def kpi_reward(cls):
27         if cls.kpi > 95:
28             print('恭喜' + cls.name + '拿到明星员工奖杯! ')
29         elif cls.kpi <= 95 and cls.kpi >= 80:
30             print('恭喜' + cls.name + '拿到优秀员工奖杯! ')
31         else:
32             print('很遗憾' + cls.name + '这次没有评上奖杯，希望来年努力工作，勇
创佳绩! ')
33
34     # 验证结果的代码
```

```
35 hrSystem.record('bob', 13000, 98)
36 hrSystem.kpi_reward()
37 hrSystem.record('candy', 8000, 89)
38 hrSystem.kpi_reward()
39 hrSystem.record('jony', 8000, 75)
40 hrSystem.kpi_reward()
```

## 2.新增检测名字的方法

### 【参考】

```
1 # 创建一个人事系统类
2 class hrSystem:
3     # 创建存储员工名字的变量 name
4     name = ''
5     # 创建存储员工工资的变量 salary
6     salary = 0
7     # 创建存储员工绩效的变量 kpi
8     kpi = 0
9
10    # 定义录入员工信息的类方法
11    @classmethod
12    def record(cls, name, salary, kpi):
13        cls.name = name
14        cls.salary = salary
15        cls.kpi = kpi
16
17    # 定义打印员工信息的类方法
18    @classmethod
19    def print_record(cls):
20        print(cls.name + '的工作信息如下: ')
21        print('本月工资: ' + str(cls.salary))
22        print('本年绩效: ' + str(cls.kpi))
23
24    # 定义根据 kpi 评奖的类方法
25    @classmethod
26    def kpi_reward(cls):
27        if cls.kpi > 95:
28            print('恭喜' + cls.name + '拿到明星员工奖杯! ')
29        elif cls.kpi <= 95 and cls.kpi >= 80:
30            print('恭喜' + cls.name + '拿到优秀员工奖杯! ')
31        else:
32            print('很遗憾' + cls.name + '这次没有评上奖杯, 希望来年努力工作, 勇
33            创佳绩! ')
34
35    # 检查录入名称是否正确的类方法
36    @classmethod
37    def check_name(cls):
38        if cls.name not in ['bob', 'candy', 'jony', 'kelly']:
39            print('录入错误! ' + cls.name + '不是本公司员工! ')
40        else:
41            print('录入正确~')
```

```
41
42 # 验证结果的代码
43 hrSystem.record('bob', 13000, 98)
44 hrSystem.check_name()
45 hrSystem.record('spy', 3000, 60)
46 hrSystem.check_name()
47
```

### 3.优化代码

#### 【参考】

```
1 # 创建一个人事系统类
2 class hrSystem:
3     # 创建存储员工名字的变量 name
4     name = ''
5     # 创建存储员工工资的变量 salary
6     salary = 0
7     # 创建存储员工绩效的变量 kpi
8     kpi = 0
9
10    # 定义录入员工信息的类方法
11    @classmethod
12    def record(cls, name, salary, kpi):
13        cls.name = name
14        cls.salary = salary
15        cls.kpi = kpi
16
17    # 定义打印员工信息的类方法
18    @classmethod
19    def print_record(cls):
20        if cls.check_name(): # 以 cls.check_name() 的返回值 (0或1) 作为判断
条件。
21            print(cls.name + '的工作信息如下: ')
22            print('本月工资: ' + str(cls.salary))
23            print('本年绩效: ' + str(cls.kpi))
24
25    # 定义根据 kpi 评奖的类方法
26    @classmethod
27    def kpi_reward(cls):
28        if cls.kpi > 95:
29            print('恭喜' + cls.name + '拿到明星员工奖杯! ')
30        elif cls.kpi <= 95 and cls.kpi >= 80:
31            print('恭喜' + cls.name + '拿到优秀员工奖杯! ')
32        else:
33            print('很遗憾' + cls.name + '这次没有评上奖杯, 希望来年努力工作, 勇
创佳绩! ')
34
35    # 检查录入名称是否正确的类方法
36    @classmethod
37    def check_name(cls):
38        if cls.name not in ['bob', 'candy', 'jony', 'kelly']:
```

```

39         print('录入错误!' + cls.name + '不是本公司员工!')
40         return 0 # 设置返回值为0
41     else:
42         print('录入正确~')
43         return 1 # 设置返回值为1
44
45 # 验证结果的代码
46 hrSystem.record('spy', 3000, 60)
47 hrSystem.print_record()

```

## 练习-日程安排-参考

### 练习目标

通过这个练习，你可以通过类方法实现简单的日程安排。

### 练习要求

为类calendar添加两个方法：一个删除完成项，一个添加新增项。

由于日程安排是事情和时间的配对，所以我们用字典来存放：

`{'给父母买礼物':'9:00', '学习':'10:00', '和朋友聚会':'18:30'}`

接下来，需要你新建两个类方法，从而实现字典中的数据增减。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 新建两种类方法

请你补全右侧的代码，让其成功运行，且结果为：

`{'给父母买礼物':'9:00', '学习':'10:00', '和朋友聚会':'18:30'}`

### 【提示】

这两个类方法的参数个数分别为2和3（删除只要键，新增要键值对）。

### 【解答】

```

1 class calendar:
2     date = '2020-08-08'
3     things = {'给父母买礼物':'9:00', '学习':'10:00', '和朋友聚会':'18:30'}
4
5
6     @classmethod
7     def thing_done(cls, thing):
8         del cls.things[thing]
9
10
11     @classmethod

```

```

12     def add_thing(cls, thing, time):
13         cls.things[thing] = time
14
15
16 calendar.thing_done('给父母买礼物')
17 calendar.add_thing('写日记', '20:00')
18 print(calendar.things)
19
20

```

## 第13关

### 练习-问卷搜集器-参考

#### 1. 问卷答案的储存

##### 【参考】

```

1 # 不用运行，读懂即可。
2 class Survey():
3     # 收集调查问卷的答案
4     def __init__(self, question):
5         self.question = question
6         # 定义收集问卷问题答案的列表
7         self.response = []
8
9     # 显示调查问卷的题目
10    def show_question(self):
11        print(self.question)
12
13    # 存储问卷搜集的答案
14    def store_response(self, new_response):
15        self.response.append(new_response)
16        # 将答案添加到列表self.response（见第7行

```

#### 2. 搜集问题-最喜欢的美食

##### 【参考】

```

1 class Survey():
2     # 收集调查问卷的答案
3     def __init__(self, question):
4         self.question = question
5         self.response = []
6
7     # 显示调查问卷的题目
8     def show_question(self):
9         print(self.question)
10
11    # 存储问卷搜集的答案

```

```

12     def store_response(self, new_response):
13         self.response.append(new_response)
14
15 # 请实例化Survey()类，并且显示出这次的调查问卷问题。
16 food_survey = Survey('你最喜欢的美食是什么? ') # 类的实例化，同时为参数
question赋值。
17 food_survey.show_question() # 调用类方法打印问题
18
19 # 存储问卷调查的答案
20 while True:
21     response = input('请回答问卷问题，按 q 键退出: ')
22     if response == 'q':
23         break
24     food_survey.store_response(response) # 再次调用类方法，进行问答的收集。
25
26 # 输出测试
27 for food in food_survey.response:
28     print('美食: ' + food)

```

### 3.实名问卷：搜集名字和籍贯地

#### 【参考】

```

1 class Survey():
2     # 收集调查问卷的答案
3     def __init__(self, question):
4         self.question = question
5         self.response = []
6
7     # 显示调查问卷的题目
8     def show_question(self):
9         print(self.question)
10
11     # 存储问卷搜集的答案
12     def store_response(self, new_response):
13         self.response.append(new_response)
14
15 # 请定义实名调查问卷的新类 RealNameSurvey，继承自 Survey 类
16 class RealNameSurvey(Survey):
17     def __init__(self, question):
18         Survey.__init__(self, question)
19         self.response = {} # 由于籍贯地和名字挂钩，所以用构成为“键值对”的字典来
存放。
20
21     # 存储问卷搜集的答案（覆盖父类的类方法）
22     def store_response(self, name, new_response): # 除了 self，还需要两个参
数。
23         self.response[name] = new_response # 键值对的新增
24
25 survey = RealNameSurvey('你的籍贯地是哪? ')
26 survey.show_question()
27 while True:

```



```

28     response = input('请回答问卷问题，按 q 键退出：')
29     if response == 'q':
30         break
31     name = input('请输入回答者姓名：')
32     survey.store_response(name, response) # 调用类方法，将两次通过 input 的
    字符串存入字典。
33
34 # 输出测试
35 for name, value in survey.response.items():
36     print(name + ': ' + value)
37

```

## 练习-那个男人来了-参考

### 练习目标：

这个练习，会训练你去运用“子类的继承和定制”相关知识。

### 练习要求：

在练习中，需要你通过代码的运行结果和所学知识，补全代码。

在补全代码的过程中，需要对子类的继承和定制有清晰的认知。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 根据结果，倒推代码

请你补充右侧代码，让其运行结果如下：

大家注意了！

一个叫“吉多”的人来了。

大家注意了！

一个叫“范罗苏姆”的男人来了。

大家注意了！

那个叫“范罗苏姆”的男人留下了他的背影。

### 【提示】

几个提示：

- 1.每次实例化都会打印“大家注意了！”，所以可以把它放在初始化函数中；
- 2.父类的实例化有传入参数，子类的不用，说明继承时改造了初始化函数；
- 3.子类比父类多了一种方法（因父类生成的对象无法调用），可知这个方法是在子类中新增的。

### 【解答】

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4         print('大家注意了！')
5

```

```

6
7     def show(self):
8         print('一个叫“%s”的人来了。' % self.name)
9
10
11 class Man(Person):
12     def __init__(self):
13         Person.__init__(self, name='范罗苏姆')
14
15     def show(self):
16         print('一个叫“%s”的男人来了。' % self.name)
17
18
19     def leave(self): # 子类定制新方法
20         print('那个叫“%s”的男人留下了他的背影。' % self.name)
21
22
23 author1 = Person('吉多')
24 author1.show()
25 author2 = Man()
26 author2.show()
27 author3 = Man()
28 author3.leave()
29
30

```

## 第14关

### 练习-升级版游戏-参考

#### 1.代码实验

##### 【参考】

```

1 import random
2
3 player_list = ['【圣光骑士】', '【暗影刺客】', '【精灵弩手】']
4 players = []
5
6 for i in range(3):
7     player = random.choice(player_list)
8     players.append(player)
9
10 print(players)
11
12 # 判断方式1: 比较运算符
13
14 # 角色类型都一样
15 if players[0] == players[1] == players[2]:

```

```

16     print('我们都一样! —方式1') # 在打印结果中加上“方式N”，验证不同的判断方式的
    有效性。
17
18 # 角色类型都不一样
19 if players[0] != players[1] and players[0] != players[2] and players[1] !=
    players[2]:
20     print('我们都不一样! —方式1')
21
22 # 判断方式2: set()
23 # 集合（set）是一个无序的不重复元素序列，set()可以去重，然后生成一个集合。
24
25 players_set = set(players)
26
27 # 角色类型都一样&角色类型都不一样
28 if len(players_set) == 1:
29     print('我们都一样! —方式2')
30 elif len(players_set) == 3:
31     print('我们都不一样! —方式2')
32
33 # 其他判断方式
34
35 '''
36 除了上面两种方式外，还有很多方式可以考虑：
37
38 例如：类似判断方式2，可以新建一个列表，用 append 添加不重复的角色类型，然后看这个列
    表的长度是多少（类似判断方式2）。
39 再如：“角色类型完全不一样”，可先将players和player_list按相同的排序方式排一下，再判
    断两个排序后的列表是否相同。
40
41
42 总而言之：只要你愿意思考、尝试和搜索，就可以得出各种不同的判断方式。
43 '''
44
45 # 再举一个例子：
46 if sorted(players) == sorted(player_list):
47     print('我们都不一样! —方式3')
48 else:
49     pass
50

```

## 2.加新功能

### 【参考】

```

1 # 查看新增的代码和注释
2
3
4 import random
5 import time
6
7
8 # 创建一个类，可实例化成具体的游戏角色

```

```

9 class Role:
10     def __init__(self, name):
11         self.name = name
12         self.life = random.randint(100,150)
13         self.attack = random.randint(30,50)
14
15
16 # 创建3个子类，可实例化为3个不同的角色类型
17
18
19 class Knight(Role):
20     def __init__(self, name = '【圣光骑士】'):
21         Role.__init__(self,name)
22         self.life = int(self.life*1.5)
23         self.attack = int(self.attack*0.8)
24
25
26     def fight_buff(self, opponent, str1, str2):
27         if opponent.name == '【暗影刺客】':
28             self.attack = int(self.attack * 1.5)
29             print('【%s】【圣光骑士】对 【%s】【暗影刺客】说：“让无尽光芒制裁你的
30 堕落！”'%(str1, str2))
31
32
33 class Assassin(Role):
34     def __init__(self, name = '【暗影刺客】'):
35         Role.__init__(self,name)
36         self.life = int(self.life*0.8)
37         self.attack = int(self.attack*1.5)
38
39
40 # 角色类型克制关系
41
42     def fight_buff(self, opponent, str1, str2):
43         if opponent.name == '【精灵弩手】':
44             self.attack = int(self.attack * 1.5)
45             print('【%s】【暗影刺客】对 【%s】【精灵弩手】说：“主动找死，就别怪我
46 心狠手辣。”'%(str1, str2))
47
48
49 class Bowman(Role):
50     def __init__(self, name = '【精灵弩手】'):
51         Role.__init__(self,name)
52         self.life = int(self.life*1.2)
53         self.attack = int(self.attack*1.2)
54
55
56     def fight_buff(self, opponent, str1, str2):
57         if opponent.name == '【圣光骑士】':
58             self.attack = int(self.attack * 1.5)
59             print('【%s】【精灵弩手】对 【%s】【圣光骑士】说：“骑着倔驴又如何？你
60 都碰不到我衣服。”'%(str1, str2))

```

```

57
58
59 # 创建一个类，可生成3V3并展示：可分为：欢迎语→随机生成→展示角色
60 class Game:
61     def __init__(self):
62         self.players = []
63         self.enemies = []
64         self.score = 0
65         self.i = 0
66         # 依次执行以下函数
67         self.game_start() # 欢迎语
68         self.born_role() # 随机生成6个角色
69         self.cooperat_role() # 角色类型引起的属性加成
70         self.show_role() # 展示角色
71         self.order_role() # 排序并展示
72         self.pk_role() # 让双方 Pk 并展示结果
73         self.show_result() # 展示最终结局
74
75
76 # 欢迎语
77 def game_start(self):
78     print('----- 欢迎来到“炼狱角斗场” -----')
79     print('在昔日的黄昏山脉，奥卢帝国的北境边界上，有传说中的“炼狱角斗场”。')
80     print('鲜血与战斗是角斗士的归宿，金钱与荣耀是角斗士的信仰！')
81     print('今日，只要你【你的队伍】能取得胜利，你将获得一笔够花500年的财富。')
82     time.sleep(2)
83     print('将随机生成【你的队伍】和【敌人队伍】！')
84     input('\n狭路相逢勇者胜，请按任意键继续。\\n')
85
86 # 随机生成6个角色
87 def born_role(self):
88     for i in range(3):
89
90 self.players.append(random.choice([Knight(),Assassin(),Bowman()]))
91
92 self.enemies.append(random.choice([Knight(),Assassin(),Bowman()]))
93
94 # 判断是否满足角色类型配合的条件
95 def cooperat_role(self):
96     players_list =
[ self.players[0].name,self.players[1].name,self.players[2].name]
97     enemies_list =
[ self.enemies[0].name,self.enemies[1].name,self.enemies[2].name]
98     players_set = set(players_list)
99     enemies_set = set(enemies_list)
100     # print(players_list) # 这几行是验证代码，正式代码里会删掉。感兴趣的
101     # print(players_set)
102     # print(enemies_list)
103     # print(enemies_set)

```

```

103         if len(players_set) == 1:
104             print('我方触发了“角色类型一致”的条件，每个角色的血量增加25%。')
105             for i in range(3):
106                 self.players[i].life = int(self.players[i].life * 1.25)
107         if len(players_set) == 3:
108             print('我方触发了“角色类型兼备”的条件，每个角色的攻击增加25%。')
109             for i in range(3):
110                 self.players[i].attack = int(self.players[i].attack *
111 1.25)
112         if len(enemies_set) == 1:
113             print('敌方触发了“角色类型一致”的条件，每个角色的血量增加25%。')
114             for i in range(3):
115                 self.enemies[i].life = int(self.enemies[i].life * 1.25)
116         if len(enemies_set) == 3:
117             print('敌方触发了“角色类型兼备”的条件，每个角色的攻击增加25%。')
118             for i in range(3):
119                 self.enemies[i].attack = int(self.enemies[i].attack *
120 1.25)
121
122         input('请按任意键，查看【你的队伍】和【敌人队伍】的角色信息：') # 缓一
123         缓，再展示。
124
125         # 展示角色
126         def show_role(self):
127             print('----- 角色信息 -----')
128             print('你的队伍：')
129             for i in range(3):
130                 print('『我方』 %s 血量： %s 攻击： %s'%
131 (self.players[i].name,self.players[i].life,self.players[i].attack))
132             print('-----')
133
134             print('敌人队伍：')
135             for i in range(3):
136                 print('『敌方』 %s 血量： %s 攻击： %s'%
137 (self.enemies[i].name,self.enemies[i].life,self.enemies[i].attack))
138             print('-----')
139             input('请按回车键继续。\\n')
140
141         # 排序并展示
142         def order_role(self):
143             order_dict = {}
144             i=0
145             while i < 3:
146                 order = int(input('你想将 %s 排在第几个上场？（输入数字1-3）'%
147 (self.players[i].name)))
148                 if order in [1,2,3]:
149                     if order in order_dict:

```

```

148         print('你输入了重复的数字, 请重新输入: ')
149     else:
150         order_dict[order] = self.players[i]
151         i += 1
152     else:
153         print('输入有误, 请输入1或2或3: ')
154     self.players = []
155     for i in range(1,4):
156         self.players.append(order_dict[i])
157     print('\n你的队伍出场顺序是: %s、%s、%s'
158           %(self.players[0].name,self.players[1].name,self.players[2].name))
159     print('敌人队伍出场顺序是: %s、%s、%s'
160           %(self.enemies[0].name,self.enemies[1].name,self.enemies[2].name))
161
162
163 # 让双方 Pk 并展示结果
164 def pk_role(self):
165     for i in range(3):
166         print('\n----- 【第%s轮】 -----' %
167               (i+1))
168         # 每一局开战前加buff
169         self.players[i].fight_buff(self.enemies[i], '我方', '敌方')
170         self.enemies[i].fight_buff(self.players[i], '敌方', '我方')
171         input('\n战斗双方准备完毕, 请按回车键继续。')
172         print('-----')
173         while self.players[i].life > 0 and self.enemies[i].life > 0:
174             self.enemies[i].life -= self.players[i].attack
175             self.players[i].life -= self.enemies[i].attack
176             print('我方%s 发起了攻击, 敌方%s 剩余血量 %s'%
177                   (self.players[i].name,self.enemies[i].name,self.enemies[i].life))
178             print('敌方%s 发起了攻击, 我方%s 剩余血量 %s'%
179                   (self.enemies[i].name,self.players[i].name,self.players[i].life))
180             print('-----')
181             time.sleep(1)
182             if self.players[i].life <= 0 and self.enemies[i].life > 0:
183                 print('\n很遗憾, 我方%s 挂掉了! '% (self.players[i].name))
184                 self.score -= 1
185             elif self.players[i].life > 0 and self.enemies[i].life <= 0:
186                 print('\n恭喜, 我方%s 活下来了。'% (self.players[i].name))
187                 self.score += 1
188             else:
189                 print('\n我的天, 他们俩都死了啊! ')
190
191
192 # 展示最终结局
193 def show_result(self):
194     input('\n请按回车查看最终结果。\\n')
195     if self.score > 0:

```

```
196         print('【最终结果】\n你赢了，最终的财宝都归你了！')
197     elif self.score == 0:
198         print('【最终结果】\n你没有胜利，但也没有失败，在夜色中灰溜溜离开了奥
    卢帝国。')
199     else:
200         print('【最终结果】\n你输了。炼狱角斗场又多了几具枯骨。')
201
202
203 game = Game()
204
```

## 练习-优化游戏体验-参考

### 练习目标

通过这个练习，优化项目3中排序部分的代码。

### 练习要求

项目3中的排序，3次输入必须包含1、2、3，否则就会报错（下面可以体验）。

为了避免由于输错而产生的报错，我们可以在原有代码的基础上做一些优化，让程序自动提示，并且能够多次输入，直到成功排序为止。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 发现问题

请你运行代码3次，在排序时分别输入：

第一次输入：1→2→3

第二次输入：1→2→2

第三次输入：4→5→6

后面两种输入会报错，你可以想一想为什么会报错以及如何避免这个错误。

### 解决方案

对于解决方案，你有了大致的思路了吗？

想一想以下的场景，你应该会有所启发：

你在注册某APP账号时，假设密码不符合要求，会跳出一个提示说：密码只能是……

当你两次输入的密码不一致时，又会跳出一个提示说：你输入的密码不一致。

同理，我们可以优化一下排序的代码：

当输入的不是123中的某个数字，就跳出提示；当输入的是重复的数字，也跳出提示。

这样，问题得解决，代码得优化。

### 【提示】

1. 要保证每个数字都输入了，可用 while 循环判断：每输入一个123中不重复的数字，就记一次有效，需要有效3次；

2. 要验证某个元素是否在列表里，可用 'if a in list 1:'。



### 【解答】

```
1 import random
2
3
4 player_list = ['【狂血战士】','【森林箭手】','【光明骑士】','【独行剑
   客】','【格斗大师】','【枪弹专家】']
5 players = random.sample(player_list,3)
6 order_dict = {}
7
8
9 i=0
10 while i < 3:
11     order = int(input('你想将 %s 排在第几个上场？（输入数字1-3）'%
   (players[i])))
12     if order in [1,2,3]:
13         if order in order_dict:
14             print('你输入了重复的数字，请重新输入：')
15         else:
16             order_dict[order] = players[i]
17             i += 1
18     else:
19         print('输入有误，请输入1或2或3：') # 报错2
20
21
22 players = []
23 for i in range(1,4):
24     players.append(order_dict[i])
25 print('\n我方角色的出场顺序是： %s %s %s\n' %
   (players[0],players[1],players[2]))
26
27
```

## 第15关

### 练习-数据转移中的变化-参考

#### 1.编码和解码

##### 【参考】

```
1 # 1.分别使用gbk和utf-8编码自己的名字，并将其打印出来。
2 print('吴枫'.encode('gbk'))
3 print('吴枫'.encode('utf-8'))
4
5 # 2.复制上一步得到的结果，进行解码，打印出你的名字（两次）。
6 print(b'\xe5\x90\xb4\xe6\x9e\xab'.decode('utf-8'))
```

```

7
8 # 3.使用gbk解码
   b'\xb7\xe7\xb1\xe4\xbf\xc6\xbc\xbc\xd3\xd0\xd2\xe2\xcb\xbc', 并打印出来。
9
10 print(b'\xb7\xe7\xb1\xe4\xbf\xc6\xbc\xbc\xd3\xd0\xd2\xe2\xcb\xbc'.decode('
   gbk'))

```

## 2.通过文件读写，复制图片

### 【参考】

```

1 # 先打开图片
2 with open('photo1.png','rb') as file: # 以“rb”模式打开图片
3     data = file.read()
4     with open('photo3.png','wb') as newfile: # 以“wb”模式写入
5         newfile.write(data)

```

## 2.在读写之间处理数据-2

### 【参考】

```

1 file1 = open('scores.txt','r',encoding='utf-8')
2 file_lines = file1.readlines()
3 file1.close()
4
5 final_scores = []
6
7 for i in file_lines:
8     data = i.split()
9     sum = 0 # 先把总成绩设为0
10    for score in data[1:]: # 遍历列表中第1个数据和之后的数据
11        sum = sum + int(score) # 然后依次加起来，但分数是字符串，所以要转换
12
13    result = data[0]+str(sum)+'\n' # 结果就是学生姓名和总分
14    print(result)
15    final_scores.append(result)
16
17 print(final_scores)
18
19 sum1 = open('winner.txt','w',encoding='utf-8')
20 sum1.writelines(final_scores)
21 sum1.close()

```

## 2.在读写之间处理数据-2

### 【参考】

```

1 ile1 = open('winner.txt','r',encoding='utf-8')
2 file_lines = file1.readlines()
3 file1.close()
4
5 dict_scores = {}
6 list_scores = []

```

```

7  final_scores = []
8
9  # print(file_lines)
10 # print(len('\n'))
11
12 # 打印结果为: ['罗恩102\n', '哈利383\n', '赫敏570\n', '马尔福275\n']
13 # 经过测试,发现'\n'的长度是1。所以,名字是“第0位-倒数第5位”,分数是“倒数第4位-倒
    数第二位”。
14 # 再根据“左取右不取”,可知: name-[:-4],score-[-4:-1]
15
16 for i in file_lines: # i是字符串。
17     print(i)
18     name = i[:-4] # 取出名字(注:字符串和列表一样,是通过偏移量来获取内部数
    据。)
19     score = int(i[-4:-1]) # 取出成绩
20     print(name)
21     print(score)
22     dict_scores[score] = name # 将名字和成绩对应存为字典的键值对(注意:这里的成
    绩是键)
23     list_scores.append(score)
24
25 # print(list_scores)
26 list_scores.sort(reverse=True) # reverse, 逆行, 所以这时列表降序排列, 分数从高
    到低。
27 # print(list_scores)
28
29 for i in list_scores:
30     result = dict_scores[i] + str(i) + '\n'
31     # print(result)
32     final_scores.append(result)
33
34 print(final_scores) # 最终结果
35
36 winner_new = open('winner_new.txt','w',encoding='utf-8')
37 winner_new.writelines(final_scores)
38 winner_new.close()
39

```

## 练习-古诗默写-参考

### 练习目标

这个练习,会让你学会一种方法,可以直接修改原文件中的数据。

### 练习要求

语文老师将一些古诗存在txt文档里,一句一行。

最近,他计划抽一些古诗,自己设置一些空来让学生默写。

请你用代码帮老师完成这项工作(只要处理了一个文档,加上循环就能处理无数个文档了)。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

我们以李商隐的《锦瑟》为例，这是原档里的内容：

锦瑟

[唐] 李商隐

锦瑟无端五十弦，  
一弦一柱思华年。  
庄生晓梦迷蝴蝶，  
望帝春心托杜鹃。  
沧海月明珠有泪，  
蓝田日暖玉生烟。  
此情可待成追忆，  
只是当时已惘然。

老师在这首诗想考学生“一弦一柱思华年。”和“只是当时已惘然。”，即他想得到的是：

锦瑟

[唐] 李商隐

锦瑟无端五十弦，  
\_\_\_\_\_。  
庄生晓梦迷蝴蝶，  
望帝春心托杜鹃。  
沧海月明珠有泪，  
蓝田日暖玉生烟。  
此情可待成追忆，  
\_\_\_\_\_。

### 【提示】

为了让代码逻辑简洁且便于更新，可以将需要默写的诗句都放到一个表格里。

另外，当遇到默写诗句时，可以用英文的下划线去替代（\_\_\_\_\_）。

### 【解答】

```
1 list_test = ['一弦一柱思华年。\\n','只是当时已惘然。\\n']
2 with open('poem2.txt','r') as f:
3     lines = f.readlines()
4 print(lines)
5 with open('poem2.txt','w') as new:
6     for line in lines:
7         if line in list_test:
8             new.write('_____。\\n')
9         else:
10            new.write(line)
11
12
```

## 第16关

### 练习-时间记录器-参考

#### 【参考】

```
1 import time
2
3 input("欢迎使用“时间管理器”！ 请按回车继续。")
4
5 while True:
6     task_name = input('请输入任务名: ')
7     task_time = int(input('你觉得自己至少可以专注这个任务多少分钟？ 输入 N 分钟'))
8     input('此次任务信息: \n我要完成的任务: %s\n我至少要专注: %d分钟\n按回车开始计时:'%(task_name,task_time))
9     start = time.time() # 开始计时
10    start_time = time.strftime('%Y-%m-%d %H:%M:%S',time.localtime(time.time())) # 格式化日期
11
12    # for t in range(task_time*60,0,-1): # 实际代码: 分钟转成秒要乘60, 用-1来倒计时。
13    for t in range(task_time,0,-1):
14        info = '请专注任务, 还要保持专注 ' + str(t) + ' 秒哦! '
15        print(info,end="")
16        print("\b"*(len(info)*2),end="",flush=True)
17        time.sleep(1)
18    print('你已经专注了 %d 分钟, 很棒~再加把劲, 完成任务! '%task_time) # 倒计时后, 才继续运行之后的代码。
19
20
21    # 询问任务是否完成
22    task_status = input('请在任务完成后按输入y:')
23    if task_status == 'y':
24        end = time.time() # 一定用户按了 y, 就记下结束时间。
25        end_time = time.strftime('%Y-%m-%d %H:%M:%S',time.localtime(time.time())) # 日期格式化
26        actual_time = int((end -start)/60) # 始末时间相减, 从秒换算到分, 除以60。
27
28        start_end = start_time + '—' + end_time + '\n'
29        with open('time\log3.txt','a', encoding = 'utf-8') as f:
30            f.write(task_name + ' 的预计时长为: ' + str(task_time) + '分钟\n')
31            f.write(task_name + ' 的实际时长为: ' + str(actual_time) + '分钟,具体时间为: ' + start_end)
32        again = input('建立一个新任务请按 y, 退出时间日志记录器请按 q: ')
33        if again == 'q':
34            break
```

```
34     else:
35         print('抱歉，你的输入有误。请重启时间记录器。')
36
37 print('愿被你善待的时光，予你美好的回赠。')
38
```

## 练习-再出古诗默写题-参考

### 练习目标

通过这个练习，我们会用模块去实现上一关卡的选做题“古诗默写”。

### 练习要求

这个练习，我们会接触一个新的Python内置模块：os（文件/目录方法）。这个模块中的许多方法，配合文件读写以及数据处理，可以让一些工作得以自动化。当然，在一个练习里，我们不会奢求那么多，先体验一下os模块里的3个方法即可。

#### 【讲解】

明确目标很重要（所以重复三遍）。  
做到后面的步骤，可再点开左侧的“练习介绍”查看。

我们先回顾一下不用模块是如何出古诗默写题的

#### 【提示】

os 模块中的替换方法  
可能你会觉得这么操作更麻烦，但假设要你处理大量的文档，模块会让你的代码更清晰更简洁。  
细心一些，你可以的！

#### 【解答】

```
1 import os
2
3 list_test = ['一弦一柱思华年。\\n','只是当时已惘然。\\n']
4
5 with open ('poem2.txt','r') as f:
6
7     lines = f.readlines()
8
9 with open('poem2_new.txt','w') as new:
10
11     for line in lines:
12
13         if line in list_test:
14
15             new.write('_____。\\n')
16
17     else:
```

```
18
19         new.write(line)
20
21 os.replace('poem_new.txt', 'poem1.txt')
```

另外，建议你在本地编辑器试一试下面的这些方法：

```
os.getcwd() # 返回当前工作目录
os.listdir(path) # 返回path指定的文件夹包含的文件或文件夹的名字的列表
os.mkdir(path) # 创建文件夹
os.path.abspath(path) # 返回绝对路径
os.path.basename(path) # 返回文件名
os.path.isfile(path) # 判断路径是否为文件
os.path.isdir(path) # 判断路径是否为目录
```

注：dir为单词directory（目录）的缩写。

## 第17关

### 练习-自制动态二维码-参考

#### 【参考】

```
1 # 先导入模块
2 from MyQR import myqr
3
4
5 myqr.run(
6     words='http://weixin.qq.com/r/kzlje9TEE4lsrZAY92yB',
7     # 扫描二维码后，显示的内容，或是跳转的链接
8     version=5, # 设置容错率
9     level='H', # 控制纠错水平，范围是L、M、Q、H，从左到右依次升高
10    picture='she-3.gif', # 图片所在目录，可以是动图
11    colorized=True, # 黑白(False)还是彩色(True)
12    contrast=1.0, # 用以调节图片的对比度，1.0 表示原始图片。默认为1.0。
13    brightness=1.0, # 用来调节图片的亮度，用法同上。
14    save_name='Python.gif', # 控制输出文件名，格式可以是 .jpg, .png, .bmp
15    , .gif
16    )
```

### 练习-绝对值的多种求法-参考

#### 练习目标：

通过这个练习，你至少可以用3种方法去求某个数的绝对值。

### 练习要求：

请你通过搜索和步骤中的提示，用不同的方法求绝对值。

建议将其封装成函数，如“abs\_value1()”。

### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 挑战3种方法解题

请根据右侧的关键词提示以及网上搜索，用3种不同的方法求绝对值。

### 【提示】

内置函数为 abs()，内置模块为 math。

注：绝对值的英文为 absolute value。

### 【解答】

```
1  import math
2
3
4  def abs_value1():
5      a = float(input('1.请输入一个数字: '))
6      if a >= 0:
7          a = a
8      else:
9          a = -a
10     print('绝对值为: %f' % a)
11
12
13 def abs_value2():
14     a = float(input('2.请输入一个数字: '))
15     a = abs(a)
16     print('绝对值为: %f' % a)
17
18
19 def abs_value3():
20     a = float(input('3.请输入一个数字: '))
21     a = math.fabs(a)
22     print('绝对值为: %f' % a)
23
24
25 abs_value1()
26 abs_value2()
27 abs_value3()
28
29
```



## 第18关

### 练习-拯救选择困难症-参考

#### 【参考】

```
1  # 帮你做选择之我要吃什么
2  import random
3
4
5  # 将需要用到的表格和变量放在开头
6  list_food = ['KFC', '蒸菜馆', '楼下快餐店', '桂林米粉', '东北饺子', '金牌猪脚
7  饭', '三及第汤饭'] # 备选菜单，可自定义。
8  list_choice = []
9
10 # 由于两个原因都包含判断过程，所以，为了让代码更简洁，可将其封装成函数。
11 def choose(list):
12     while True:
13         food = random.choice(list)
14         judgement = input('去吃【%s】好不好啊？同意的话输入y，不想吃直接回车即
15 可。'%(food))
16         if judgement == 'y':
17             print('去吃【%s】！就这么愉快地决定啦！'%(food))
18             break
19
20 # 判断环节
21 reason = int(input('你不知道吃什么的原因是：1.完全不知道吃什么；2.在几家店之间徘
22 徊（请输入1或2）：'))
23 if reason == 1:
24     choose(list_food)
25 elif reason == 2:
26     add = True
27     while add:
28         choice = input('请输入让你犹豫的店名（注：一家一家输，完成后输入y）：')
29         if choice != 'y': # 这个判断语句，是为了不将 y 也添加到菜单里。
30             list_choice.append(choice)
31         if choice == 'y':
32             add = False
33         choose(list_choice)
34 else:
35     print('抱歉，目前还不支持第三种情况——不过，你可以加代码哦。')
36
```

## 练习-模拟广告牌-参考

### 练习目标

我们会通过今天的作业，用代码模拟一种现实中的场景：滚动的广告牌。

### 练习要求

实际生活中，有些广告牌是单行滚动的字体。

这个练习会通过新旧知识的结合，用Python实现对这种效果的模拟。

#### 【讲解】

明确目标很重要（所以重复三遍）。

做到后面的步骤，可再点开左侧的“练习介绍”查看。

### 提出产品需求，形成技术方案

由于是练习，所以是没有让每个人根据自己的生活经验来提需求。

而是直接给定了产品需求：用Python代码模拟单行的滚动广告。

初步形成的技术方案是：print+字符串+循环+time模块（控制滚动速度）。

#### 【讲解】

在这个需求下，稍稍分析即可得：

广告显示：打印字符串；

广告滚动：字符串的改变和循环（注：字符串和列表一样，可以用偏移量来提取数据）。另外，循环的速度影响滚动速度，可以用 time.sleep() 来控制。

为了让这种模拟和实际的更接近，补充一个模块中的方法。

请你试着运行右侧的代码，然后将其注释掉，再用代码完成对广告牌的模拟，广告词可以自己起。

对了，正常的滚动广告是无限循环的。不过，你可以在代码中设置循环次数，只要保证效果即可。

#### 【提示】

关键代码：

```
`content = content[1:] + content[0]`
```

#### 【解答】

```
1 import os, time
2
3 def main():
4
5     content = ' 风变编程，陪你一起学Python ' # 广告词可自定义。
6
7     while True:
8
9         os.system('clear')
10
11         print(content)
12
13         content = content[1:] + content[0]
```

```
14
15     time.sleep(2)
16
17 if __name__ == '__main__': # 类里面学到的检测方法，在函数中其实也可以用。
18
19     main()
```